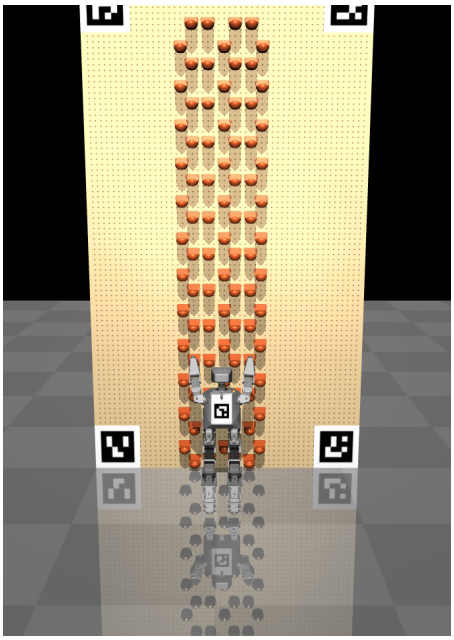
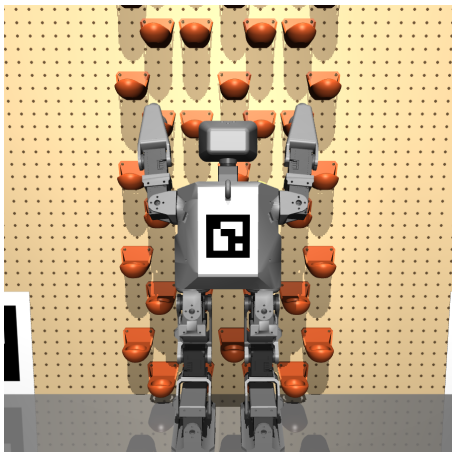
Perception par marqueurs ArUco

Hackathon Rock Climbing Robots – règlement v1.1, section 6.1

Où sont les caméras ? Où est le robot ? Où sont les prises ?
Ce qui a été fait, comment ça marche, comment s'en servir





29 octobre 2026

Document de travail de l'équipe. Code dans `Hackathon/perception/`.

Table des matières

1 En bref **3**

2 Le contexte **3**

2.1 Ce que dit le règlement . . . 3
2.2 Le simulateur . . . 4
2.3 Organisation des dossiers . . . 4
2.4 Installation . . . 5

3 Les repères **5**

4 Les marqueurs ArUco **6**

4.1 Qu'est-ce qu'un marqueur ArUco ? . . . 6
4.2 La détection : `detect()` . . . 7

5 Étape 1 — Où sont les caméras ? **8**

5.1 Le modèle de la caméra . . . 8
5.2 Le problème PnP . . . 8
5.3 Le code : `camera_pose()` . . . 8

6 Étape 2 — Le marqueur ID 5 sur le robot **9**

6.1 La fiche des organisateurs . . . 9
6.2 Le repère du marqueur dans le repère du tronc . . . 10
6.3 L'ajout au simulateur : `MarkedSim` . . . 10

7 Étape 3 — Où est le robot ? **11**

7.1 La chaîne des repères . . . 11
7.2 La triangulation . . . 11
7.3 Deux caméras obligatoires . . . 11
7.4 Le petit marqueur : 1920×1080 . . . 12
7.5 L'inclinaison par l'IMU : `level_with_gravity()` . . . 12
7.6 Du marqueur au tronc : `robot_pose()` . . . 13

8 Étape 4 — Où sont les prises ? **13**

8.1 L'idée . . . 13
8.2 Classer les pixels . . . 13
8.3 Le score d'une place . . . 14
8.4 Choisir les prises . . . 14
8.5 Le cumul au fil des images . . . 15
8.6 La couleur, et le résultat . . . 15

9 Tout ensemble : la classe Tracker **16**

10 Les programmes et leurs commandes **17**

10.1 `demo.py` — la démonstration dans le simulateur 17

10.2 `live.py` — le jour J (vraies caméras ou images) 19

10.3 `doc/make_figures.py` 20

11 Comment tester **20**

12 Résultats **21**

12.1 Pose des caméras 21

12.2 Suivi du robot, marqueur officiel de 42 mm 21

12.3 Les prises, dans le simulateur 22

12.4 Les prises, sur une photo du vrai mur 22

13 Pour le vrai robot **23**

14 Limites et prochaines étapes **24**

14.1 Limites connues 24

14.2 Prochaines étapes 25

A Aide-mémoire des commandes **25**

B Référence des fonctions **26**

C Petit glossaire **26**

1 En bref

Le règlement (section 6.1) propose un mode de perception *avec marqueurs ArUco* : quatre marqueurs fixés aux coins du mur donnent la position des caméras, et un cinquième marqueur, collé sur le robot, donne la position du robot. C'est ce mode qui a été programmé et testé dans le simulateur MuJoCo des organisateurs, avec en plus la détection des prises.

Ce qui marche aujourd'hui

1. **Les caméras se repèrent seules** : dans chaque image, le code trouve les marqueurs ID 1 à 4 des coins du mur et en déduit où est la caméra et où elle regarde, *sans qu'on le lui dise*. Précision : environ **2 mm** sur sa position, **0,02°** sur son orientation.
2. **Le robot porte le marqueur ID 5 officiel** : l'autocollant de la fiche des organisateurs (code de 42 mm, à plat sur le dos), reproduit dans le simulateur.
3. **Le robot est localisé** en croisant les deux caméras (en 1920 × 1080) et en prenant son inclinaison dans sa centrale inertielle (IMU) : erreur moyenne de **0,9 à 1,6 mm** (pire cas 4,3 mm), sur les murs N1, N2, N3 et pour des caméras placées au hasard.
4. **Les prises sont trouvées** par leur couleur, calées sur la grille de trous : **100 %** des prises sur le mur vide, aucune fausse ; les prises cachées par le robot apparaissent dès qu'il bouge. Sur une **photo du vrai mur** : les **52 prises**, avec leur couleur.
5. **Le même code servira sur le vrai robot** : un script (`live.py`) lit les caméras USB, se calibre sur les marqueurs du mur, trouve les prises et suit le robot en direct.

Ce qui n'est pas encore fait. Le robot ne grimpe pas encore : la démonstration le fait seulement se balancer sur ses prises de départ. Il reste à écrire le programme de grimpe, à relier l'IMU du vrai robot à `live.py`, et à tout essayer avec les vraies caméras (section 14).

Les commandes à connaître (à lancer depuis le dossier `Hackathon/simulator`) :

```
# 1. demonstration dans le simulateur, avec une fenetre (mur N1, N2 ou N3)
uv run --with opencv-contrib-python python ../perception/demo.py N2 --show
# 2. la video de la demonstration
open ../perception/out/demo.mp4
# 3. la meme chose avec les cameras placees au hasard (le nombre change le placement)
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 3
# 4. le jour J : les deux cameras USB (gauche puis droite)
uv run --with opencv-contrib-python python ../perception/live.py 0 1
```

2 Le contexte

2.1 Ce que dit le règlement

— **Section 5.1** — deux caméras USB par mur (Intel RealSense D435i), une de chaque côté, *fournies et placées par les organisateurs*. Les équipes branchent les câbles sur leur ordinateur et récupèrent les images. **Seule l'image couleur (RGB) est autorisée** (pas la profondeur).

— **Section 5.2** — quatre marqueurs aux coins du mur, IDs 1 à 4, dictionnaire `DICT_4X4_50`. Chacun : code noir de 100 × 100 mm entouré d'une bordure blanche de 18 mm (plaque de 136 × 136 mm), le coin extérieur de la plaque sur le coin du mur. L'équipe peut ajouter

un marqueur **ID 5 sur son robot** ; les organisateurs en fournissent une fiche à imprimer (section 6).

— **Section 6.1** — avec les marqueurs, les marqueurs du mur donnent la pose (position + orientation) des caméras par rapport au mur ; le robot est suivi grâce à son marqueur ID 5 (ou détecté autrement).

— **Section 8** — utiliser les marqueurs multiplie le temps par **1,25** (le mode sans marqueur, plus difficile, garde ×1). La hauteur atteinte compte avant le temps : un robot fiable avec marqueurs bat un robot sans marqueur qui tombe.

2.2 Le simulateur

Le simulateur des organisateurs (`rock-climbing-robots/simulator`, paquet Python `climbing_sim`) contient le robot Microban et ses 19 moteurs, le mur de 800 × 1400 mm avec ses prises, les quatre marqueurs des coins et deux caméras simulées qui ont les caractéristiques des D435i. Il ne contient pas de marqueur sur le robot : on l'ajoute nous-mêmes (section 6).

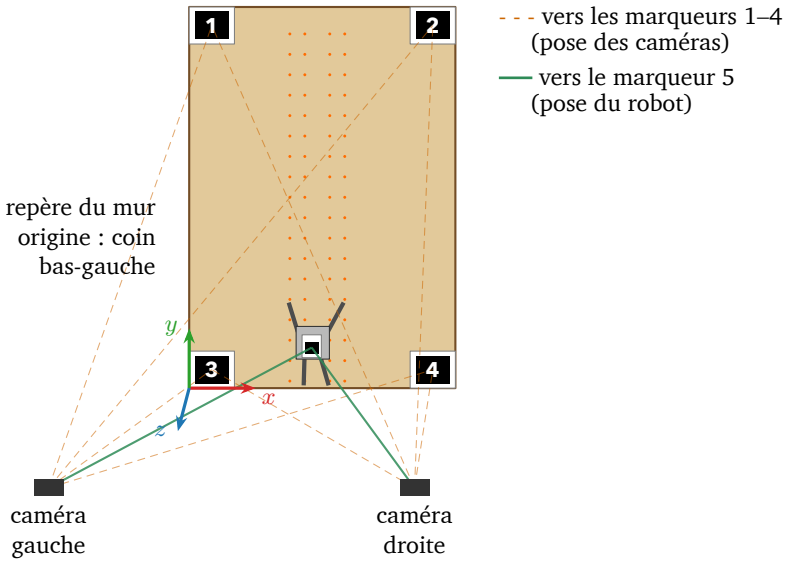


FIGURE 1 – Le dispositif (schéma, mur N1). Les deux caméras voient les quatre marqueurs des coins, ce qui leur permet de se situer ; elles voient aussi le marqueur ID 5 sur le dos du robot, ce qui permet de situer le robot, et les prises (en orange), qui sont repérées par leur couleur.

2.3 Organisation des dossiers

Tout est dans `~/Desktop/Hackathon` (dépôt GitHub `Alyaa203/Hackathon`). Les dépôts des organisateurs (`simulator`) et de Rhoban (`microban`) n'y sont pas recopiés : ce sont des *sous-modules*, des liens vers la version exacte utilisée. Ils ne sont **pas modifiés** : notre code est à part, dans `perception`.

```
Hackathon/
|-- simulator/          depot des organisateurs (climbing_sim) -- sous-module, non
    modifie
|-- microban/           programme du vrai robot (Rhoban, branche dev_v2) -- sous-module
`-- perception/         NOTRE CODE
    |-- perception.py     le traitement : marqueurs, cameras, robot, prises
    |-- robot_marker.py   le marqueur ID 5 du robot (fiche des organisateurs) +
    simulateur
    |-- demo.py           demonstration dans le simulateur, comparee a la verite
```

```
|-- live.py           le jour J : les vraies cameras USB (ou des images)
|-- out/              produit par demo.py et live.py : demo.mp4, holds.json, images
`-- doc/              ce document (perception_aruco.tex) et ses images
```

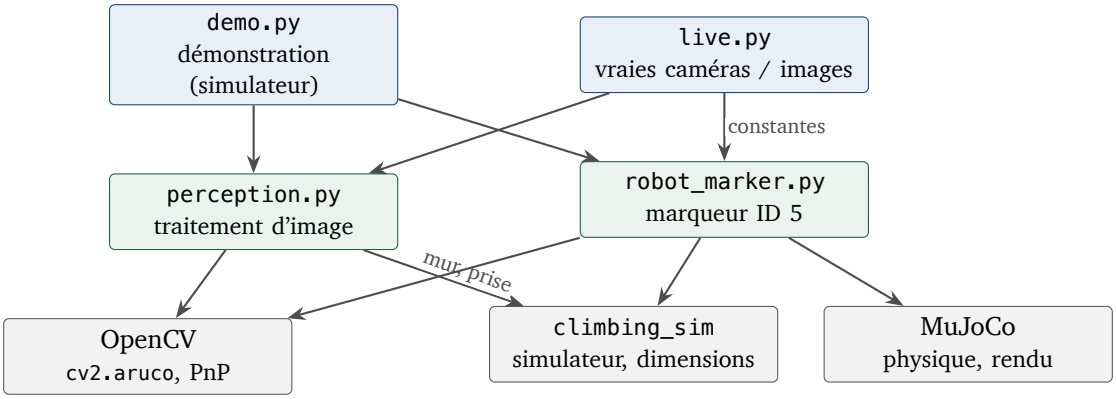


FIGURE 2 – Qui utilise quoi. `perception.py` ne dépend que d'OpenCV et des dimensions du mur et de la prise (données du simulateur) : il fonctionne aussi bien sur des images simulées que sur des images réelles.

2.4 Installation

Il faut `git`, `git-lfs` et `uv`. OpenCV n'est *pas* ajouté au projet du simulateur (pour ne pas modifier le dépôt des organisateurs) : on le demande à chaque lancement avec `--with`. `uv` le télécharge la première fois (une minute) puis le garde en cache.

```
git clone --recurse-submodules https://github.com/Alyaa203/Hackathon
cd Hackathon/simulator && uv sync
uv run --with opencv-contrib-python python ../perception/demo.py N1
```

Le message « VIRTUAL_ENV ... does not match »

Il apparaît quand un autre environnement Python (`tf_env`) est activé dans le terminal. Il est sans conséquence (`uv` utilise bien l'environnement du simulateur). Pour le faire disparaître : `deactivate`.

3 Les repères

Toute la difficulté de la localisation est de passer d'un repère à l'autre. Il y en a cinq.

Repère	Axes	Utilisé par
Mur $\mathcal{W}$	origine au coin bas-gauche de la face avant ; x à droite, y vers le haut, z sort du mur vers le grimpeur	marqueurs, caméras, résultats
Monde (MuJoCo)	même origine ; x à droite, y *entre dans* le mur, z vers le haut	simulateur, planification
Caméra $\mathcal{C}$	convention OpenCV : x à droite de l'image, y vers le bas, z devant la caméra	OpenCV
Marqueur $\mathcal{M}$	centre du code ; x à droite, y vers le haut (vu de face), z sort du marqueur	OpenCV
Tronc $\mathcal{T}$	x devant le robot (vers le mur quand il grimpe), y à sa gauche, z le long du dos	robot, IMU

Une *pose* (position + orientation) d'un repère $\mathcal{B}$ dans un repère $\mathcal{A}$ s'écrit avec une matrice de rotation R_{AB} (3×3) et une position t_{AB} : un point de coordonnées P_B dans $\mathcal{B}$ a pour coordonnées dans $\mathcal{A}$

$$P_A = R_{AB}\, P_B + t_{AB}.$$

Les *colonnes* de R_{AB} sont les axes de $\mathcal{B}$ exprimés dans $\mathcal{A}$. Pour passer du mur au monde du simulateur, la rotation est fixe :

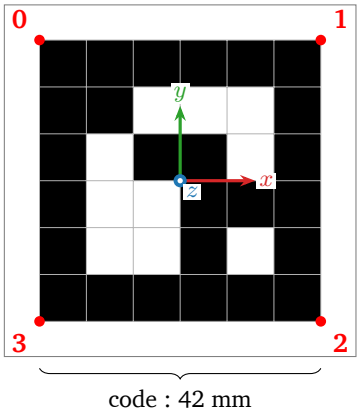
$$R_{\text{monde},\mathcal{W}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \qquad (x_{\text{monde}} = x, \quad y_{\text{monde}} = -z, \quad z_{\text{monde}} = y).$$

C'est ce que fait la fonction `to_world()`.

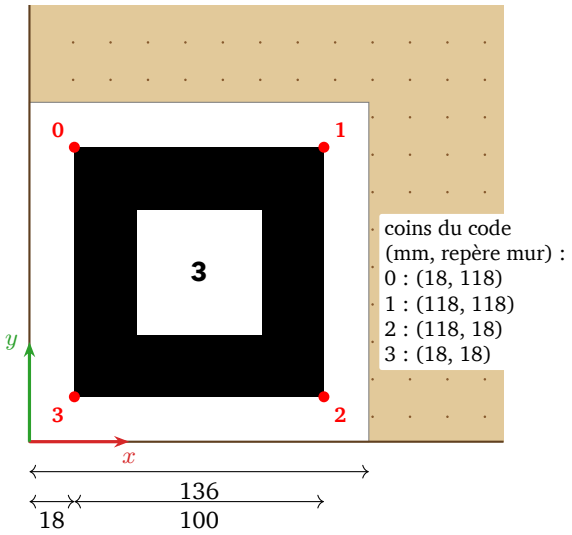
4 Les marqueurs ArUco

4.1 Qu'est-ce qu'un marqueur ArUco ?

Un marqueur ArUco est un carré noir et blanc conçu pour être retrouvé facilement par un ordinateur. Pour le dictionnaire `DICT_4X4_50` : une grille de 6×6 cases dont la couronne extérieure est noire et dont les 4×4 cases intérieures codent un numéro (ID de 0 à 49). Autour, une bordure blanche le détache du fond. Le motif n'est pas symétrique : en le lisant, OpenCV sait *quel* marqueur il voit et *dans quel sens* il est tourné, donc il peut numéroter ses quatre coins toujours dans le même ordre.



(a) Le marqueur ID 5 du robot. En rouge, l'ordre des coins rendu par OpenCV ; au centre, le repère du marqueur (z sort de la page).



(b) Le marqueur ID 3 au coin bas-gauche du mur (motif simplifié). Ses coins ont des positions *connues* dans le repère du mur.

FIGURE 3 – Anatomie d'un marqueur. Pour chaque coin, on connaît sa position en 3D (sur le mur, ou sur le marqueur) et OpenCV donne sa position en pixels dans l'image : c'est cette double information qui permet de tout localiser.

4.2 La détection : `detect()`

OpenCV fait tout le travail : l'image est passée en niveaux de gris, les quadrilatères sombres sont repérés, leur contenu est lu et comparé au dictionnaire. Deux réglages comptent :

— l'*affinage sous-pixel* des coins (`CORNER_REFINE_SUBPIX`) : la position des coins est précise à une fraction de pixel, ce qui compte beaucoup pour la précision finale ;

— la lecture de chaque case du code sur 8×8 pixels au lieu de 4×4 (`perspectiveRemovePixelPerCell = 8`) : le code du robot ne fait qu'une vingtaine de pixels de côté dans les images, et avec le réglage par défaut OpenCV le trouvait mais n'arrivait pas à le lire sur environ une image sur six (25 images sur 31 lues pour une caméra) ; avec ce réglage, il le lit à chaque fois (31 sur 31).

```
_params = cv2.aruco.DetectorParameters()
_params.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX
_params.perspectiveRemovePixelPerCell = 8
_detector = cv2.aruco.ArucoDetector(cv2.aruco.getPredefinedDictionary(cv2.aruco.
    DICT_4X4_50), _params)

def detect(image_rgb) -> dict[int, np.ndarray]:
    gray = cv2.cvtColor(np.ascontiguousarray(image_rgb), cv2.COLOR_RGB2GRAY)
    corners, ids, _ = _detector.detectMarkers(gray)
    if ids is None:
        return {}
    return {int(i): c.reshape(4, 2) for i, c in zip(ids.ravel(), corners)}
```

Le résultat est un dictionnaire `{ID: coins}`, chaque entrée contenant les 4 coins (4×2 pixels) dans l'ordre de la figure 3. Les positions 3D des coins des marqueurs du mur sont calculées par `wall_marker_corners(id)` à partir des dimensions du règlement. La face avant des plaques est à $z = 1$ mm du mur dans le simulateur (plaques de 1 mm d'épaisseur) : constante `PLATE_Z`.

5 Étape 1 – Où sont les caméras ?

5.1 Le modèle de la caméra

Une caméra (modèle « sténopé », celui d'OpenCV) transforme un point 3D en pixel en deux temps :

1. **les extrinsèques** (R, t) — où est la caméra et où elle regarde — font passer le point du repère du mur au repère de la caméra : $P_C = R\,P_W + t$;
2. **les intrinsèques** K — propres à la caméra (focale f_x, f_y et centre optique c_x, c_y en pixels) — le projettent sur l'image :

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} \propto K\,P_C, \qquad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}.$$

Les intrinsèques se mesurent une fois pour toutes (calibration de la caméra, ou valeurs du fabricant). Les extrinsèques changent dès qu'on bouge la caméra : ce sont elles qu'on cherche.

5.2 Le problème PnP

On connaît n points 3D P_i (les coins des marqueurs du mur) et leurs pixels p_i dans l'image (donnés par `detect()`). Trouver la pose (R, t) de la caméra qui explique ces pixels s'appelle le problème **PnP** (*Perspective-n-Point*) :

$$(R, t) = \arg\min_{R,t} \sum_{i=1}^{n} \left\| \pi\big(K(RP_i + t)\big) - p_i \right\|^2$$

où π divise par la troisième coordonnée. Avec les 4 marqueurs, on a $n = 16$ coins, répartis aux quatre coins du mur : c'est très contraint, donc très précis.

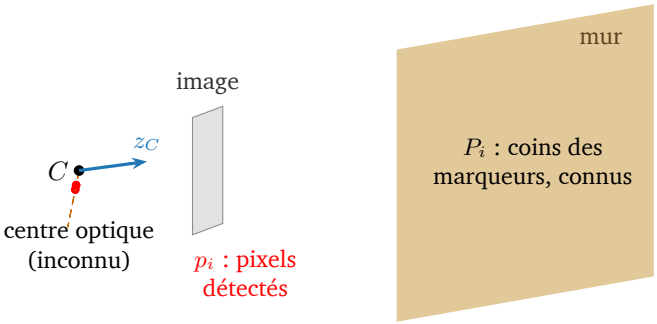


FIGURE 4 - Le problème PnP : on connaît les points du mur (P_i) et l'endroit où ils tombent dans l'image (p_i) ; on cherche la position et l'orientation de la caméra qui alignent chaque rayon.

5.3 Le code : `camera_pose()`

```python
def camera_pose(markers: dict, K, dist=None):
    ids = [i for i in WALL_IDS if i in markers]          # les marqueurs 1-4 vus
    if not ids:
        return None
    obj = np.concatenate([wall_marker_corners(i) for i in ids])  # points 3D (repere mur)
    img = np.concatenate([markers[i] for i in ids]).astype(np.float64)  # pixels
    ...
```

```
    _, rvec, tvec = cv2.solvePnP(obj, img, K, dist, flags=cv2.SOLVEPNP_IPPE)
    rvec, tvec = cv2.solvePnPRefineLM(obj, img, K, dist, rvec, tvec)
    proj, _ = cv2.projectPoints(obj, rvec, tvec, K, dist)
    rms = float(np.sqrt(np.mean(np.sum((proj.reshape(-1, 2) - img) ** 2, axis=1))))
    return rvec.ravel(), tvec.ravel(), rms
```

— `SOLVEPNP_IPPE` est une méthode exacte pour des points tous dans un même plan (ici le mur) ; elle donne une première solution, que `solvePnPRefineLM` affine par moindres carrés.

— `rvec` est la rotation R sous forme compacte (axe × angle, convertie par `cv2.Rodrigues`).

— `rms` est l'**erreur de reprojection** : on reprojette les 16 coins avec la pose trouvée et on mesure l'écart moyen avec les pixels détectés. Autour de 0,4 pixel : la pose explique très bien l'image. Plusieurs pixels : problème (mauvaises intrinsèques, marqueur mal détecté…).

— La position de la caméra dans le repère du mur s'en déduit : $C = -R^\top t$.

À retenir

Le code n'a besoin de connaître *que* les intrinsèques des caméras. Leur position est retrouvée à chaque essai à partir des marqueurs du mur : les organisateurs peuvent placer les caméras où ils veulent, tant que chacune voit les quatre coins. Il n'y a rien à mesurer au mètre.

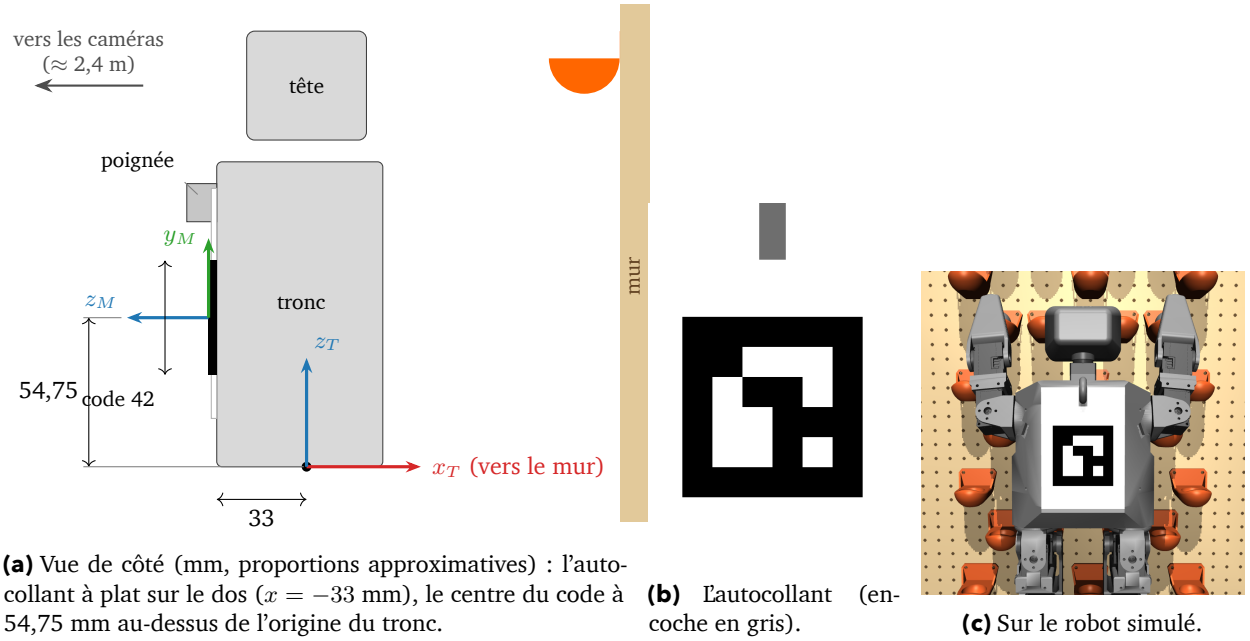
6 Étape 2 – Le marqueur ID 5 sur le robot

6.1 La fiche des organisateurs

Les organisateurs fournissent une fiche à imprimer, « *ArUco marker ID 5 (DICT_4X4_50) for the robot's back* ». Elle fixe tout : la taille du marqueur, sa place sur le robot et son repère.

Élément	Valeur (fiche des organisateurs)
Autocollant	58,0 × 84,5 mm, avec en haut une encoche de 6,19 × 13,18 mm
Code noir	**42 mm** de côté
Où le coller	à plat sur le dos du robot (poitrine + capot) ; l'encoche glisse autour du pied de la poignée jusqu'en butée, ce qui le centre et le met à la bonne hauteur ; les côtés tombent sur les bords du dos, le bas sur la ligne entre la poitrine et le bassin
Centre du code	$(-33{,}0;\ 0;\ 54{,}75)$ mm dans le repère du tronc (x devant, y à gauche, z en haut)
Axes du marqueur	$x = -y_T$ (vers la droite du robot), $y = +z_T$ (vers le haut), $z = -x_T$ (sort du dos)
Impression	à 100 % (taille réelle, pas « ajuster à la page »), vérifier la règle de 100 mm, découper le contour

Le dos du tronc a été mesuré dans le modèle du simulateur en lançant des rayons vers lui (fonction `mj_ray` de MuJoCo) : il est **plat à exactement** $-33{,}00$ **mm** sur toute la surface de l'autocollant, et la poignée dépasse jusqu'à -44 mm en haut, au centre, là où passe l'encoche. C'est cohérent avec la fiche.



(a) Vue de côté (mm, proportions approximatives) : l'autocollant à plat sur le dos ($x = -33$ mm), le centre du code à 54,75 mm au-dessus de l'origine du tronc.

(b) L'autocollant (encoche en gris).

(c) Sur le robot simulé.

FIGURE 5 – Le marqueur ID 5 des organisateurs sur le dos du robot.

6.2 Le repère du marqueur dans le repère du tronc

Pour passer de la pose du marqueur à la pose du robot, il faut savoir exactement comment le marqueur est fixé sur le tronc. La fiche le donne ; vu de derrière le robot (comme le voient les caméras), l'axe x_M va vers la droite du robot ($-y_T$), y_M vers le haut ($+z_T$), z_M vers l'arrière ($-x_T$). Ces trois vecteurs sont les colonnes de `R_TRUNK_MARKER`, et la position du centre du code est `P_TRUNK_MARKER` :

$$R_{TM} = \begin{pmatrix} 0 & 0 & -1 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \qquad t_{TM} = \begin{pmatrix} -33{,}0 \\ 0 \\ 54{,}75 \end{pmatrix} \text{ mm}, \qquad \text{côté du code : } 42 \text{ mm (`CODE`).}$$

Sur le vrai robot

Ces valeurs ne sont justes que si l'autocollant est imprimé **à 100 %** (vérifier la règle de 100 mm et le code de 42 mm) et collé comme le dit la fiche (encoche en butée contre le pied de la poignée, à plat). Un décalage de quelques millimètres se retrouve tel quel dans la position du robot. S'il est collé autrement, mesurer et corriger `P_TRUNK_MARKER` dans `robot_marker.py`.

6.3 L'ajout au simulateur : `MarkedSim`

Le simulateur construit sa scène MuJoCo avec `build_spec()`. La fonction `add_robot_marker(spec)` ajoute au corps `trunk` une boîte très fine dont la face extérieure porte l'image de l'autocollant (`sticker_image()` : le code généré par OpenCV avec `cv2.aruco.generateImageMarker`, placé comme sur la fiche, l'encoche en gris). Sa face est 0,3 mm devant le dos, pour être visible. La boîte est purement visuelle : pas de collision, pas de masse.

```
class MarkedSim(ClimbingSim):
    """ClimbingSim with the robot's marker ID 5 on its back (same arguments)."""
    def __init__(self, *args, **kwargs):
        _sim_module.build_spec = lambda **kw: add_robot_marker(build_spec(**kw))
```

```
        try:
            super().__init__(*args, **kwargs)
        finally:
            _sim_module.build_spec = build_spec
```

`MarkedSim` s'utilise exactement comme `ClimbingSim` (`MarkedSim("N1")`, `reset_hanging()`, `step()`, `render()`…). Pendant sa construction, il remplace temporairement la fonction qui fabrique la scène par une version qui ajoute l'autocollant, puis remet l'originale. Le dépôt du simulateur n'est pas modifié.

7 Étape 3 – Où est le robot ?

7.1 La chaîne des repères

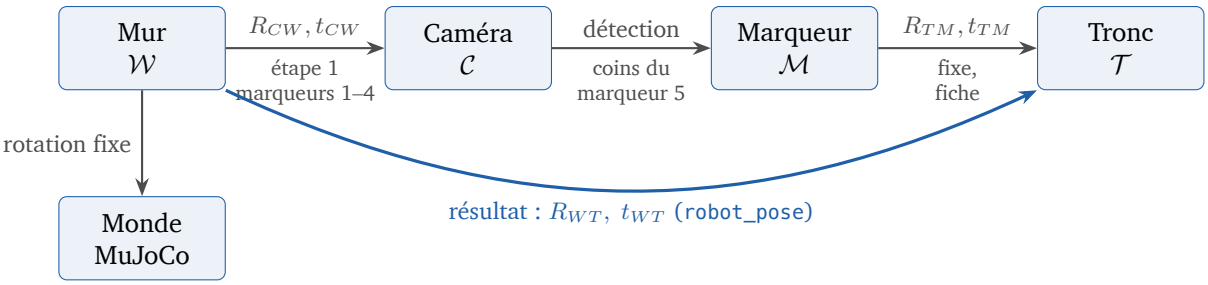


FIGURE 6 – Pour connaître la pose du tronc dans le repère du mur, on enchaîne : mur → caméra (étape 1), caméra → marqueur (détection), marqueur → tronc (fixation donnée par la fiche).

7.2 La triangulation

Chaque caméra calibrée transforme un pixel en un *rayon* dans l'espace : le coin du marqueur est quelque part sur ce rayon, mais une seule caméra ne dit pas à quelle distance. Avec deux caméras, le coin est à l'**intersection des deux rayons**. C'est la triangulation (`cv2.triangulatePoints`, méthode linéaire DLT), appliquée à chacun des 4 coins du marqueur ID 5.

On obtient 4 points 3D dans le repère du mur. On connaît aussi la forme du marqueur (un carré de 42 mm, `square_corners(0.042)`). Il reste à trouver la rotation et la translation qui superposent au mieux le carré théorique sur les 4 points mesurés : c'est l'**algorithme de Kabsch** (`_rigid_fit`), qui résout ce problème exactement avec une décomposition en valeurs singulières :

$$H = \sum_i (a_i - \bar{a})(b_i - \bar{b})^\top = U\Sigma V^\top, \qquad R = V\,\mathrm{diag}\big(1, 1, \det(VU^\top)\big)\,U^\top, \qquad t = \bar{b} - R\,\bar{a},$$

où a_i sont les coins du carré théorique et b_i les coins triangulés. Le det garantit une vraie rotation (pas un miroir).

7.3 Deux caméras obligatoires

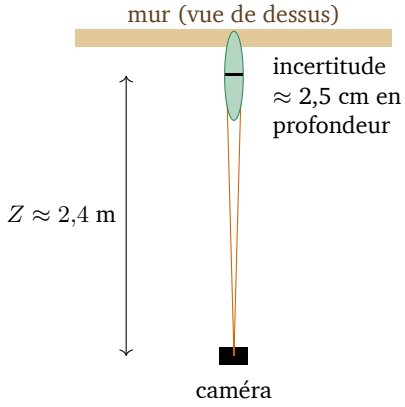
Avec une seule caméra, la distance du marqueur ne peut venir que de sa *taille apparente* dans l'image. Avec une focale f (1386 pixels pour une D435i en 1920 × 1080, 924 en 1280 × 720), un marqueur de $L = 42$ mm à $Z \approx 2{,}4$ m fait $s = fL/Z \approx 24$ pixels (16 en 1280 × 720). Une erreur δs sur sa taille donne une erreur de profondeur

$$\delta Z_{1\text{ caméra}} \approx \frac{Z^2}{f\,L}\,\delta s \approx \frac{2{,}4^2}{1386 \times 0{,}042} \times 0{,}25 \approx 2{,}5\text{ cm} \qquad \text{(3,7 cm en 1280 × 720, pour } \delta s \approx 0{,}25 \text{ pixel)}.$$

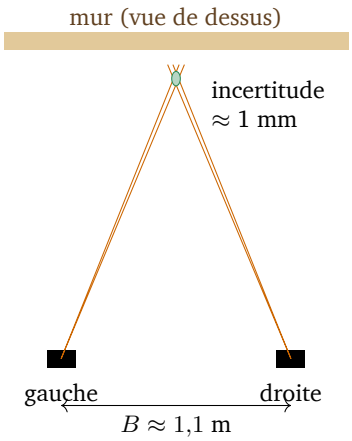
Avec deux caméras écartées de $B \approx 1{,}1$ m, c'est l'écartement qui remplace la taille du marqueur :

$$\delta Z_{2\text{ caméras}} \approx \frac{Z^2}{f\,B}\,\delta d \approx \frac{2{,}4^2}{1386 \times 1{,}1} \times 0{,}25 \approx 1\text{ mm}.$$

Le `Tracker` n'accepte donc la pose du robot que si **les deux caméras** voient son marqueur (`min_views = 2`) ; sinon `update()` renvoie `None`. Avant cette règle, les rares images vues par une seule caméra donnaient des erreurs jusqu'à **12 cm**.



(a) Une caméra : la distance vient de la *taille apparente* du marqueur (environ 24 pixels), mesure peu précise.



(b) Deux caméras : la position est à l'intersection des rayons ; l'écart B entre les caméras rend la mesure de profondeur précise.

FIGURE 7 – Pourquoi deux caméras valent beaucoup mieux qu'une.

7.4 Le petit marqueur : 1920 × 1080

Le code de 42 mm est petit : vu à 2,4 m, il fait environ 16 pixels de côté en 1280 × 720, moins de 3 pixels par case. En **1920 × 1080** (une résolution de la D435i), il en fait 24 : les coins sont mieux placés, et le marqueur est vu plus souvent par les deux caméras. C'est la résolution par défaut de `demo.py` et de `live.py` (tableau 1).

7.5 L'inclinaison par l'IMU : `level_with_gravity()`

Les coins d'un marqueur de 42 mm, placés au millimètre près, laissent une incertitude d'orientation de l'ordre de $1/21$ rad, soit 2 à 3°. Or le centre du marqueur est à $\sqrt{33^2 + 54{,}75^2} \approx 64$ mm de l'origine du tronc : 3° d'erreur d'orientation déplacent l'origine du tronc de $64 \times 0{,}05 \approx 3$ mm.

Le robot a une centrale inertielle (IMU) qui mesure la direction de la gravité dans son repère (`obs["gravity"]`), bien plus précisément. Elle donne l'*inclinaison* du robot, mais pas son *cap* (sa rotation autour de la verticale) : l'IMU ne sait pas où est le mur. On combine donc les deux :

1. avec l'orientation R donnée par le marqueur, la gravité mesurée par l'IMU (g_T, repère du tronc) devrait pointer vers le bas du mur : $v = R\,g_T$ devrait valoir $d = (0, -1, 0)$;
2. on calcule la plus petite rotation Q qui amène v sur d (axe $v \times d$, angle entre v et d), et on corrige : $R' = Q\,R$; le cap du marqueur est conservé, l'inclinaison vient de l'IMU ;
3. le centre c du marqueur, bien mesuré par la triangulation, ne bouge pas ; l'origine du tronc suit : $t' = c - R'\,t_{TM}$.

```
pose = tracker.update(images, obs["gravity"])     # marqueur + inclinaison de l'IMU
pose = tracker.update(images)                     # marqueur seul
```

Résultat : l'erreur moyenne passe de 2,4 mm à **0,9 mm** (mur N2, caméras par défaut, 1920 × 1080).

7.6 Du marqueur au tronc : `robot_pose()`

La triangulation donne la pose du marqueur dans le repère du mur, (R_{WM}, t_{WM}). Le marqueur étant fixé sur le tronc par (R_{TM}, t_{TM}), on inverse cette fixation :

$$R_{WT} = R_{WM}\, R_{TM}^{\top}, \qquad t_{WT} = t_{WM} - R_{WT}\, t_{TM}.$$

Enfin `to_world(R, p)` convertit cette pose dans le repère du simulateur, pour la comparer à la vérité (`sim.trunk_pose()`) ou pour la donner au programme de grimpe.

8 Étape 4 – Où sont les prises ?

8.1 L'idée

Les prises ne sont pas n'importe où : elles sont boulonnées sur la grille de 52 × 93 trous du mur, au pas de 15 mm, chacune sur deux trous voisins d'une même rangée. Une prise est donc désignée par sa *colonne* et sa *rangée* (celles de son trou de gauche), et il n'y a qu'environ 4 600 places possibles. On connaît aussi sa forme exacte : le modèle 3D de la vraie prise est fourni avec le simulateur. Plutôt que de chercher des taches de couleur au hasard dans l'image, on **essaie chaque place** : on projette dans chaque image la prise qui serait là, et on regarde si elle tombe sur de la couleur.

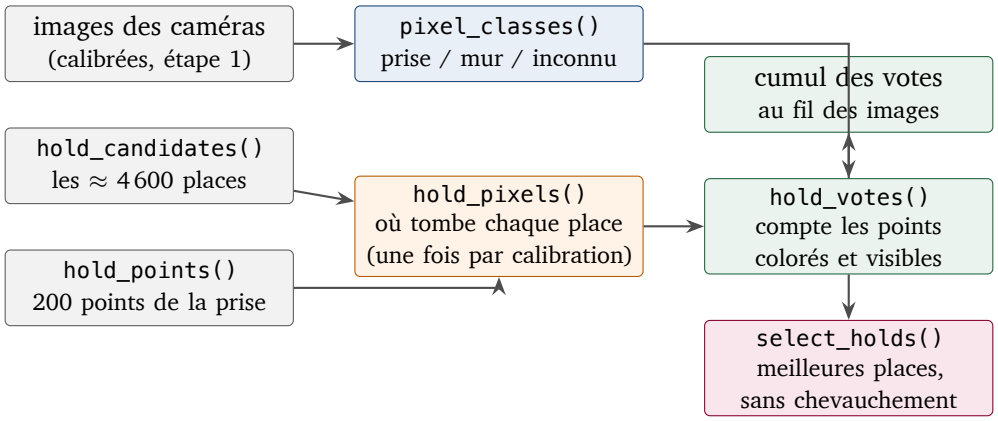


Figure 8 – La détection des prises (`Tracker.find_holds()`).

8.2 Classer les pixels

Chaque pixel est rangé dans une de trois classes, d'après sa *saturation* (la vivacité de la couleur, 0 à 255) et sa *luminosité* (0 à 255), quelle que soit sa teinte :

Classe	Règle	Ce que c'est
prise	saturation ≥ 120	une couleur vive : vert, orange, bleu… (toutes les prises)
mur	saturation entre 10 et 120	une couleur pâle : le contreplaqué (15–40 sur le vrai mur, 67 dans le simulateur)
inconnu	saturation < 10 ou luminosité < 50	gris, blanc, noir : le robot, les marqueurs, les ombres et les trous

La classe « inconnu » est essentielle : un pixel du robot *cache* peut-être une prise ; il ne doit compter ni pour ni contre. Le seuil de 10 a été fixé après un essai sur la photo du vrai mur : avec un seuil de 40, le contreplaqué réel, presque blanc, était classé « inconnu » (figure 13).

8.3 Le score d'une place

Pour chaque place possible, on prend **200 points** répartis uniformément sur la surface du modèle 3D de la prise (`hold_points()`), on les place à cet endroit du mur et on les projette dans l'image de chaque caméra. Même les points cachés (derrière la coupelle) tombent sur la prise dans l'image : la prise entière doit donc être colorée. On compte les points qui tombent sur un pixel « prise » (h), sur un pixel connu, « prise » ou « mur » (k), et dans l'image (n). Ces comptes s'additionnent sur les caméras. Alors :

$$\text{score} = \frac{h}{k} \quad \text{(la part visible qui est colorée)}, \qquad \text{vu} = \frac{k}{n} \quad \text{(la part visible, non cachée)}.$$

Une place est une prise si **score** $\geq 0{,}8$ et **vu** $\geq 0{,}4$. Ces seuils ont été choisis sur 12 cas du simulateur (3 murs, 4 placements de caméras, robot devant le mur, une seule image) :

Seuils (vu / score)	Fausses prises	Prises manquées (cachées par le robot)
0,3 / 0,7	19	65
0,3 / 0,8	7	70
0,4 / 0,8 (choisi)	**2**	**99**
0,5 / 0,8	2	121
0,6 / 0,8	3	147

(sur 984 prises au total.) Une fausse prise est bien plus dangereuse qu'une prise manquée (le robot tendrait la main dans le vide) ; et les prises manquées sont retrouvées par le cumul des images (section 8.5).

8.4 Choisir les prises

Les places sont prises de la meilleure à la moins bonne, en sautant celles dont la prise chevaucherait une prise déjà choisie (deux prises ne peuvent pas être boulonnées là ; le simulateur fournit cette règle, `OVERLAP`).

Deux prises collées. Deux prises côte à côte (à 45 mm, le plus près possible) forment une seule tache de couleur. Le score forme alors un *plateau* sur toute la largeur de la tache, avec souvent son maximum au *milieu*, entre les deux prises (figure 9) : choisir ce milieu bloquerait les deux vraies places. Après le premier choix, chaque prise choisie est donc remplacée par **deux** prises voisines si elles expliquent mieux la tache : chacune doit avoir un score $\geq 0{,}7$, et la somme de leurs excès au-dessus de 0,7 doit dépasser celui de la prise seule. Une prise isolée ne peut pas déclencher cet échange : une place décalée de 30 mm ne couvre qu'une petite partie de la prise.

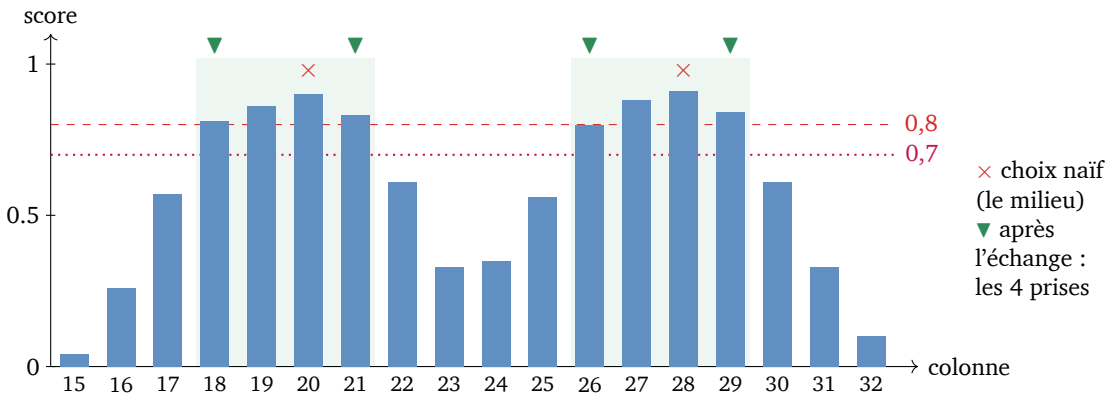


FIGURE 9 – Le score le long d'une rangée de la photo du vrai mur (rangée 9). Les deux paires de prises (colonnes 18 et 21, 26 et 29) forment deux plateaux dont le maximum est au milieu (20 et 28). L'échange « une prise → deux » retrouve les quatre vraies prises.

8.5 Le cumul au fil des images

Les prises ne bougent pas. Chaque appel à `tracker.find_holds(images)` *ajoute* les comptes h et k des nouvelles images à ceux des précédentes, et garde pour chaque place la meilleure part vue sur une image. Une prise cachée par le robot à un instant compte dès qu'elle apparaît : quand le robot monte, celles qu'il cachait en bas redeviennent visibles. `reset_holds()` oublie tout (quand les prises ont été déplacées).

La projection des 4 600 × 200 points ne dépend que de la pose des caméras : elle est faite une fois par calibration (0,3 à 0,6 s), et chaque nouvelle image ne coûte ensuite qu'environ **10 ms**.

8.6 La couleur, et le résultat

La couleur de chaque prise est la teinte moyenne de ses pixels vifs (moyenne sur un cercle, le rouge étant aux deux bouts de l'échelle des teintes), nommée : rouge, orange, jaune, vert, bleu, violet.

`find_holds()` renvoie une liste de prises (`FoundHold`), de la plus basse à la plus haute :

— `f.hold.col`, `f.hold.row` : sa place sur la grille ;

— `f.hold.socket` : le centre de sa coupelle (là où se pose la boule du robot), **repère du monde**, en mètres ; `f.socket_wall` : le même point dans le repère du mur ;

— `f.color`, `f.score`, `f.seen`.

`demo.py` et `live.py` les enregistrent dans `out/holds.json`, un fichier de mur du simulateur : on peut l'ouvrir dans l'éditeur de mur (`climbing-sim-wall-editor`) ou y accrocher le robot.

9 Tout ensemble : la classe Tracker

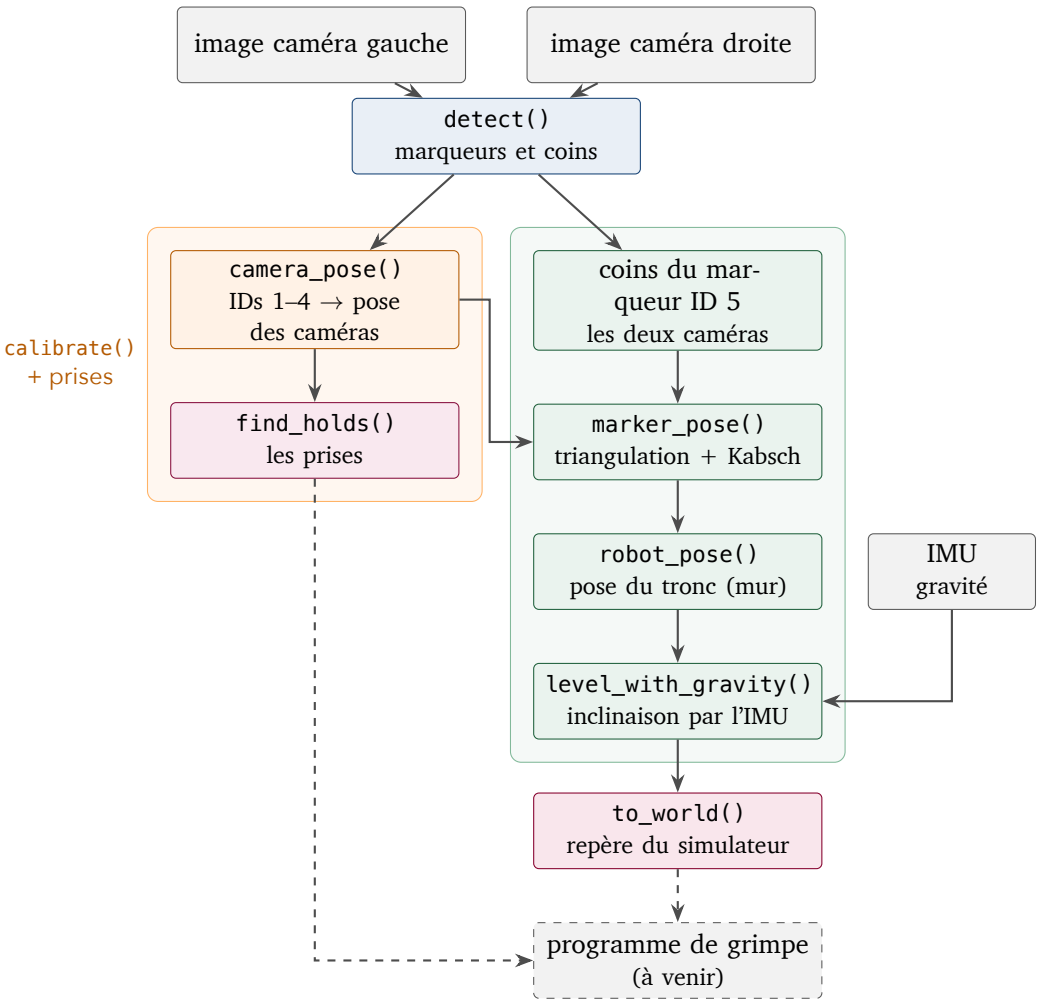


FIGURE 10 – Le traitement complet. La calibration des caméras et la recherche des prises (orange) se font au début de chaque essai, puis les prises sont complétées au fil des images ; le suivi du robot (vert, `update()`) se fait à chaque nouvelle image.

```
from perception import Tracker, to_world
from robot_marker import CODE, R_TRUNK_MARKER, P_TRUNK_MARKER

tracker = Tracker({"cam_left": K_gauche, "cam_right": K_droite},   # intrinseques
                  CODE, R_TRUNK_MARKER, P_TRUNK_MARKER)
images = {"cam_left": img_g, "cam_right": img_d}
tracker.calibrate(images)                  # debut d'essai : pose des cameras
holds = tracker.find_holds(images)         # les prises (puis de temps en temps)
pose = tracker.update(images, obs["gravity"])   # a chaque image ; None si une camera
if pose is not None:
    R, p = pose                            # tronc, repere du mur (p en metres)
    R_monde, p_monde = to_world(R, p)
for f in holds:
    print(f.hold.col, f.hold.row, f.color, f.hold.socket)   # socket : repere du monde
```

10 Les programmes et leurs commandes

Toutes les commandes se lancent depuis `~/Desktop/Hackathon/simulator` (pour que `uv` utilise l'environnement du simulateur), avec `--with opencv-contrib-python` pour ajouter OpenCV.

10.1 `demo.py` – la démonstration dans le simulateur

```
uv run --with opencv-contrib-python python ../perception/demo.py [N1|N2|N3] [--shuffle
    SEED] [--res WxH] [--show]
```

Option	Effet
`N1`, `N2`, `N3`	le mur (niveaux de difficulté 1 à 3 du règlement) ; `N1` par défaut
`--shuffle SEED`	place les deux caméras au hasard devant le mur : entre 1,9 et 3 m du mur, entre 0,5 et 1,4 m de haut, une de chaque côté, visée près du centre ; un placement est retiré tant qu'une caméra ne voit pas les quatre coins. `SEED` (un entier) choisit le tirage.
`--res WxH`	la taille des images des caméras, une résolution de la D435i : `1920x1080` par défaut, `1280x720` pour comparer
`--show`	une fenêtre montre les caméras pendant la démonstration ; à la fin, une touche la ferme

Ce que fait le script.

1. Il crée la simulation avec l'autocollant (`MarkedSim`) et accroche le robot à ses quatre prises de départ (`reset_hanging`).
2. **Les caméras** : il calibre les caméras et compare la pose trouvée à leur vraie pose.
3. **Les prises** : il les cherche, les compare aux prises du mur, en affiche quelques-unes avec leur position, et les écrit dans `out/holds.json`.
4. **Le robot** : pendant 3 secondes simulées (151 pas de 20 ms), il fait se balancer le robot (le tronc oscille de ± 30 mm de côté et ± 15 mm en hauteur, mains et pieds dans leurs prises). À chaque pas, il prend les deux images, calcule la pose du tronc avec le marqueur seul et avec l'IMU, et la compare à la vraie.
5. Il enregistre la **vidéo** `out/demo.mp4` (figure 11), et des images annotées et brutes (`cam_left.png`, `cam_left_raw.png`…).

Lire la sortie. Voici ce qu'affiche `demo.py N2` (extraits) :

```
cam_left at x -0.15 m, 0.90 m high, 2.40 m from the wall, 1920 x 1080 pixels
  cam_left: markers [1, 2, 3, 4, 5], reprojection 0.38 px, position error 1.8 mm, angle error 0.019 deg
holds from their colour (ground truth: the 88 holds of wall N2): found 81, correct 81, false 0, missed 7 (hidden
     behind the robot)
  col 18 row  2 orange  socket: world [302 -26  26] mm = on the wall x 302 mm, height 26 mm  (ok)
  all of them: /Users/.../Hackathon/perception/out/holds.json
trunk pose from marker ID 5, with the IMU's tilt (ground truth: the simulated trunk)
  t= 1.02 s: trunk at [ 386 -116  172] mm (true [ 385 -116  172]), error  0.7 mm 1.13 deg (marker alone:  6.4 mm
    )
marker alone  : mean error 2.4 mm (max 7.5), 2.33 deg (max 6.80) over 151 images
marker + IMU  : mean error 0.9 mm (max 3.0), 1.18 deg (max 5.33) over 151 images
0 of 151 images not used: the robot's marker seen by one camera only
video: /Users/.../Hackathon/perception/out/demo.mp4
```

Ligne	Signification
`cam_left at x ...`	où le simulateur a placé la caméra (la vérité, que le code ne connaît pas)
`markers [1, 2, 3, 4, 5]`	les marqueurs détectés dans l'image : il faut 1–4 (coins) et 5 (robot)
`reprojection 0.38 px`	écart moyen entre les coins détectés et les coins reprojetés : < 1 pixel = bon
`position error 1.8 mm`	distance entre la position calculée de la caméra et la vraie
`found 81, correct 81, false 0`	prises trouvées, justes, fausses ; `missed` : cachées par le robot
`socket: world [x y z]`	centre de la coupelle de la prise, repère du monde ; `on the wall` : repère du mur
`trunk at [x y z] mm`	position du tronc calculée par les caméras (avec l'IMU), repère du monde
`(true [x y z])`	la vraie position dans le simulateur
`error 0.7 mm 1.13 deg`	écart de position et d'orientation ; `marker alone` : sans l'IMU
`images not used`	images où une seule caméra voyait le marqueur du robot (pose refusée)

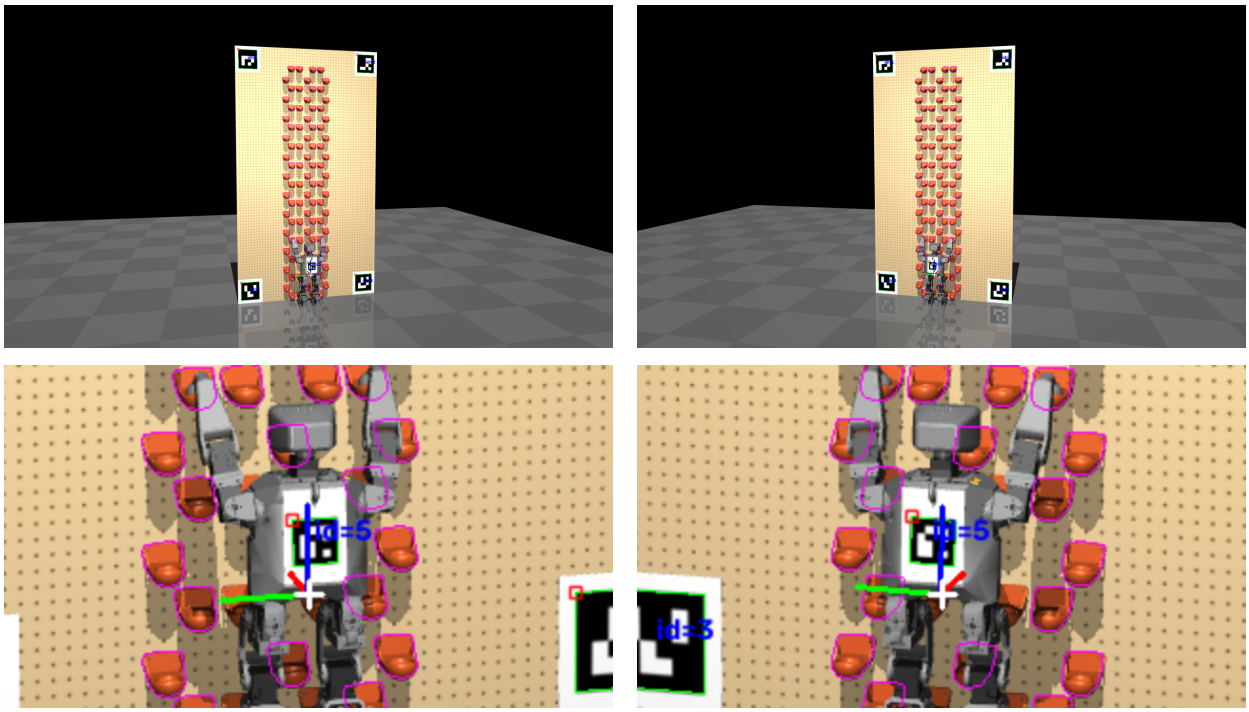


FIGURE 11 – Ce que montrent la fenêtre `--show` et la vidéo `demo.mp4` : les deux caméras (en haut) et un zoom sur le robot (en bas). En vert, les marqueurs détectés ; en magenta, le contour des prises trouvées ; les flèches rouge, verte et bleue : le repère du tronc calculé par les caméras ; la croix blanche : la vraie position du tronc. Les flèches doivent partir de la croix.

10.2 `live.py` – le jour J (vraies caméras ou images)

```
# deux cameras USB, gauche puis droite
uv run --with opencv-contrib-python python ../perception/live.py 0 1
# avec la calibration des cameras
uv run --with opencv-contrib-python python ../perception/live.py 0 1 --cameras cams.json
# sur deux images (ici les images brutes laissees par la demo), sans fenetre
uv run --with opencv-contrib-python python ../perception/live.py \
    ../perception/out/cam_left_raw.png ../perception/out/cam_right_raw.png --once
```

Argument	Effet
`0 1` (sources)	une ou deux sources, **la caméra gauche en premier** : un numéro de caméra USB ou un fichier image. Une RealSense apparaît comme plusieurs caméras (couleur, infrarouge…) : essayer 0, 1, 2… jusqu'à voir l'image couleur.
`--size 1920x1080`	la taille d'image demandée aux caméras (par défaut 1920 × 1080, pour le petit marqueur du robot)
`--cameras cams.json`	la calibration de chaque caméra (`fx`, `fy`, `cx`, `cy`, et facultativement `dist`, les coefficients de distorsion d'OpenCV). Sans ce fichier : les valeurs théoriques de la D435i (moins précis).
`--once`	une seule image par caméra, pas de fenêtre : affiche la liste des prises et la position du robot, enregistre `out/holds.json` et les images annotées

Les touches dans la fenêtre. **c** : recalibrer les caméras et rechercher les prises (au début de chaque essai, de préférence *avant* de mettre le robot sur le mur) ; **h** : afficher la liste des prises dans le terminal et l'écrire dans `out/holds.json` ; **s** : sauvegarder les images annotées ; **q** : quitter.

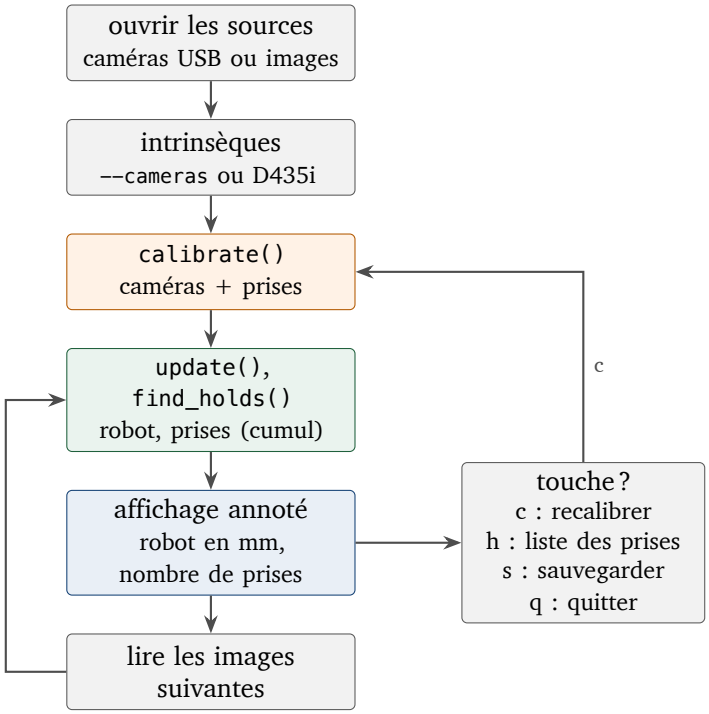


FIGURE 12 – La boucle de `live.py`.

La ligne d'état indique par exemple :

```
robot: x    391 mm, height    172 mm,   116 mm from the wall, seen by ['cam_left', 'cam_right']
robot: marker seen by cam_left only (both cameras needed: one is centimetres off)
```

Les positions sont dans le **repère du mur** : x depuis le bord gauche, hauteur depuis le bas, distance au mur. Avec **h**, la liste des prises :

```
81 holds, from the bottom up (grid column and row; their socket: x from the wall's left edge, height from its
     bottom):
  col 18 row  2  orange  x  302 mm, height    26 mm
  col 21 row  2  orange  x  348 mm, height    26 mm
```

10.3 `doc/make_figures.py`

Régénère les images de ce document à partir du simulateur (`perception/doc/img/`). Le document se compile avec `latexmk -xelatex perception_aruco.tex` dans `perception/doc/`.

11 Comment tester

1. **La démonstration, avec la fenêtre.**

   ```
   cd ~/Desktop/Hackathon/simulator
   uv run --with opencv-contrib-python python ../perception/demo.py N2 --show
   ```

   Elle dure environ 30 secondes ; à la fin, cliquer dans la fenêtre et appuyer sur une touche. C'est réussi si : chaque caméra voit `[1, 2, 3, 4, 5]` avec une reprojection sous 1 pixel ;

`false 0` pour les prises ; `marker + IMU : mean error` autour de 1 mm ; peu ou pas d'images inutilisées ; dans la fenêtre, les flèches partent de la croix blanche.

2. **La vidéo**, à revoir autant qu'on veut :

```
open ../perception/out/demo.mp4
```

3. **Les autres murs, d'autres placements de caméras, l'autre résolution** :

```
uv run --with opencv-contrib-python python ../perception/demo.py N3
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 3
uv run --with opencv-contrib-python python ../perception/demo.py N2 --res 1280x720
```

4. **Les prises trouvées**, dans l'éditeur de mur du simulateur (dans le navigateur) :

```
uv run climbing-sim-wall-editor ../perception/out/holds.json
```

5. **Le script du jour J sur des images** (sans caméra), avec les images brutes laissées par la démo :

```
uv run --with opencv-contrib-python python ../perception/live.py \
    ../perception/out/cam_left_raw.png ../perception/out/cam_right_raw.png --once
```

12 Résultats

Toutes les mesures du simulateur comparent la sortie du code à la vérité du simulateur. La simulation est déterministe : les mêmes commandes redonnent exactement les mêmes chiffres.

12.1 Pose des caméras

Caméra (1920 × 1080)	Marqueurs vus	Reprojection	Erreur de position	Erreur d'angle
gauche	1, 2, 3, 4, 5	0,38 px	1,8 mm	0,019°
droite	1, 2, 3, 4, 5	0,38 px	1,8 mm	0,020°

Avec des caméras placées au hasard (`--shuffle`), elles sont localisées à 2 à 8 mm près : plus elles sont loin et de biais, moins c'est précis.

12.2 Suivi du robot, marqueur officiel de 42 mm

Mur	Caméras	Marqueur seul (mm)		**Marqueur + IMU** (mm)	
		moyenne	max	moyenne	max
N1	par défaut	2,6	5,9	**1,1**	2,6
N2	par défaut	2,4	7,5	**0,9**	3,0
N3	par défaut	2,4	7,5	**0,9**	3,0
N2	`--shuffle 1`	3,6	9,0	**1,1**	3,2
N2	`--shuffle 2`	2,9	6,1	**1,4**	4,3
N2	`--shuffle 3`	3,5	10,1	**1,6**	4,3
N2	`--shuffle 4`	2,6	7,8	**1,0**	2,5
N2	`--shuffle 5`	2,2	9,0	**1,0**	3,0
N2	par défaut, 1280 × 720	3,8	10,0	**1,6**	3,9

En 1920 × 1080 (sauf la dernière ligne), sur 151 images chacun ; au plus 1 image sur 151 inutilisée (une seule caméra). L'erreur d'orientation moyenne est de 1,2 à 2,8° avec l'IMU (2,3 à 4,1° sans).

Comment on en est arrivé là. Avec le marqueur officiel, la toute première version (1280 × 720, marqueur seul, une caméra acceptée) se trompait de 5 à 15 mm en moyenne, avec des pics à **120 mm**. Les réglages ont été comparés sur 4 placements de caméras et 204 images (avant le réglage du détecteur de la section 4.2) :

Résolution	Images vues par une seule caméra	Marqueur seul	Marqueur + IMU
1280 × 720	26 sur 204	4,6 mm (max 12,9)	2,4 mm (max 6,4)
1920 × 1080	14 sur 204	3,1 mm (max 9,9)	**1,3 mm (max 4,3)**

TABLE 1 – Le choix des réglages (moyenne et maximum de l'erreur de position, images vues par les deux caméras). Le réglage du détecteur a ensuite ramené les images vues par une seule caméra à 0 ou 1 sur 151.

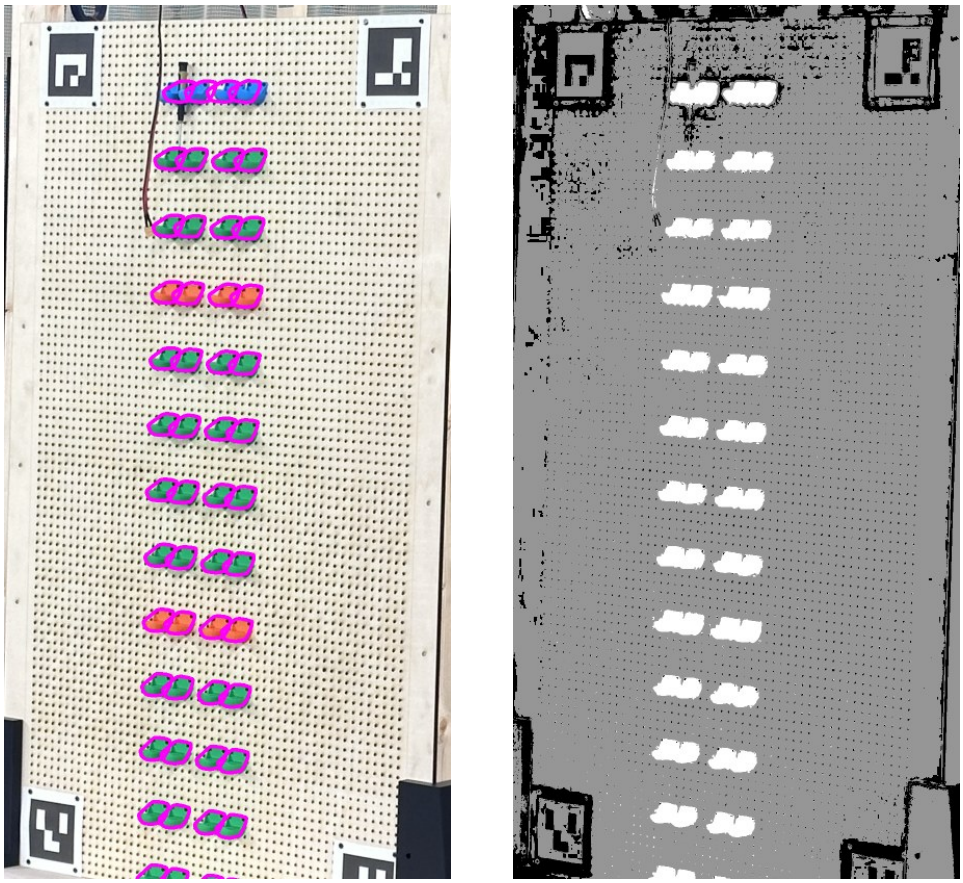
12.3 Les prises, dans le simulateur

	N1 (72 prises)	N2 (88 prises)	N3 (86 prises)
Robot devant, une seule image	64 justes, 0 fausse	81 justes, 0 fausse	78 justes, 1 fausse
Mur vide (avant de poser le robot)	**72 / 72, 0 fausse**	**88 / 88, 0 fausse**	**86 / 86, 0 fausse**
Cumul, robot monté de 20 cm	70 justes, 0 fausse	87 justes, 0 fausse	84 justes, 0 fausse
Cumul, robot monté de 40 cm	**72 / 72, 0 fausse**	**88 / 88, 0 fausse**	**86 / 86, 0 fausse**

(Caméras par défaut. « Cumul » : le robot accroché de plus en plus haut, les votes de chaque image ajoutés.) Les prises manquées sont toutes cachées derrière le robot. Temps : 0,3 à 0,6 s la première fois, puis environ 10 ms par image.

12.4 Les prises, sur une photo du vrai mur

Le code a été essayé sur une photo du vrai mur prise au téléphone, de loin et très de biais. Les réglages de l'appareil n'étant pas connus, sa focale a été estimée à partir des 16 coins des marqueurs (`cv2.calibrateCamera`, une seule image : 6 622 pixels, reprojection 2,9 pixels).



(a) Les 52 prises trouvées (contours magenta).

(b) Les classes : blanc = prise, gris = mur, noir = inconnu.

FIGURE 13 – La détection des prises sur la photo du vrai mur (recadrée sur le mur).

— **52 prises trouvées** : 13 rangées de 4, aux rangées 2, 9, 16… 86 (une rangée tous les 105 mm) et aux colonnes **18, 21, 26 et 29** : l'échelle du mur N2, prises de mains à l'extérieur, de pieds à l'intérieur.

— **Les couleurs sont justes** : bleu pour la rangée du haut (86), orange pour les rangées 30 et 65, vert pour les autres — exactement la photo.

— **À vérifier** : la rangée bleue du haut sort aux colonnes 19, 22, 25 et 28, décalée d'une colonne vers l'intérieur, alors qu'elle semble alignée sur les autres. L'accroche de la corde passe devant, et la focale du téléphone n'est qu'estimée.

— **Ce que l'essai a corrigé** : d'abord 42 prises, car le contreplaqué réel, presque blanc, était classé « inconnu » (seuil de gris abaissé de 40 à 10) ; puis 28, une par paire, au milieu des paires collées (échange « une prise → deux ») ; enfin 52.

13 Pour le vrai robot

1. **Brancher les deux caméras** en USB sur l'ordinateur de l'équipe. Les organisateurs les placent ; il ne faut pas les déplacer, et rien n'est à mesurer.
2. **Trouver leurs numéros** : lancer `live.py 0 1`, puis 1 2, 2 3… jusqu'à voir les deux images couleur. **Seule l'image couleur (RGB) est autorisée** par le règlement.
3. **Calibrer les intrinsèques** de chaque caméra (une fois), **en 1920 × 1080** : soit lire les valeurs d'usine de la RealSense (`rs2_intrinsics`), soit faire une calibration OpenCV avec un damier.

Les écrire dans un fichier `cams.json` :

```
{"cameras": [
  {"name": "cam_left",  "width": 1920, "height": 1080,
   "fx": 1365.4, "fy": 1364.8, "cx": 961.9, "cy": 543.0, "dist": [0, 0, 0, 0, 0]},
  {"name": "cam_right", "width": 1920, "height": 1080,
   "fx": 1368.7, "fy": 1367.9, "cx": 956.7, "cy": 539.1, "dist": [0, 0, 0, 0, 0]}]}
```

(valeurs d'exemple). La clé `dist` n'est lue que par `live.py` : garder un fichier séparé pour l'éditeur de caméras du simulateur.

4. **Imprimer la fiche du marqueur ID 5** à 100 %, vérifier la règle de 100 mm et le code de 42 mm, découper le contour, coller l'autocollant à plat sur le dos, l'encoche en butée contre le pied de la poignée.
5. **Vérifier la calibration** avec l'éditeur de caméras du simulateur : y charger une photo de la vraie caméra et les paramètres trouvés ; le mur dessiné par-dessus doit tomber exactement sur le vrai mur.

   ```
   uv run climbing-sim-cameras
   ```

6. **À chaque essai**, *avant* de mettre le robot sur le mur : appuyer sur **c** (caméras et prises), vérifier que chaque caméra voit `[1, 2, 3, 4]` avec une reprojection inférieure à 1 pixel, et avec **h** que la liste des prises est complète et sans fausse prise.
7. **Les couleurs** : si une prise réelle est mal vue (couleur pâle, éclairage), ajuster les seuils `HOLD_SAT_MIN` et `GREY_SAT_MAX` en tête de la partie « holds » de `perception.py`.

Ce qui n'a pas pu être testé

La lecture de vraies caméras USB n'a pas été testée : pas de matériel, et la webcam de l'ordinateur n'a pas été ouverte. Le premier essai sur place doit vérifier le numéro des caméras, la résolution (1920 × 1080) et que l'image reçue est bien l'image couleur. La détection des prises n'a été essayée que sur une photo de téléphone.

14 Limites et prochaines étapes

14.1 Limites connues

— **L'IMU du vrai robot n'est pas encore reliée à `live.py`** : sur le vrai robot, le suivi utilise le marqueur seul (environ 2,5 mm en moyenne dans le simulateur, contre 1 mm avec l'IMU). Il faudra transmettre la gravité mesurée par le robot (`obs["gravity"]`) à `tracker.update()`.

— **Occultation** : si un bras passe devant le marqueur dans une caméra, `update()` renvoie `None` (une caméra ne suffit pas). Le programme de grimpe doit le prévoir (garder la dernière pose connue, ou utiliser les angles des moteurs).

— **Prises cachées** : une prise que le robot cache aux deux caméras n'est pas vue tant qu'il est devant ; d'où l'intérêt de chercher les prises avant de mettre le robot sur le mur, puis de cumuler.

— **Simulateur idéal** : images sans flou, sans bruit, sans distorsion, éclairage parfait. Les vraies images seront moins nettes : la précision réelle sera moins bonne que ces chiffres.

— **Pénalité** : le mode avec marqueurs multiplie le temps par 1,25.

14.2 Prochaines étapes

1. **Écrire le programme de grimpe** : bouger un membre à la fois en gardant les trois autres accrochés, avec la cinématique inverse du simulateur, en s'appuyant sur la position du tronc et des prises données par le `Tracker`.
2. **Relier l'IMU du vrai robot** à `live.py`.
3. **Essayer avec les vraies caméras** et le vrai mur, et ajuster les seuils de couleur si besoin.

A Aide-mémoire des commandes

```
cd ~/Desktop/Hackathon/simulator          # toujours depuis ce dossier

# demonstration (mur N1, N2 ou N3), avec une fenetre ; la video
uv run --with opencv-contrib-python python ../perception/demo.py N2 --show
open ../perception/out/demo.mp4
# cameras au hasard, autre resolution
uv run --with opencv-contrib-python python ../perception/demo.py N2 --shuffle 3
uv run --with opencv-contrib-python python ../perception/demo.py N2 --res 1280x720

# les prises trouvees, dans l'editeur de mur
uv run climbing-sim-wall-editor ../perception/out/holds.json

# le jour J : cameras USB (gauche puis droite), avec ou sans calibration
uv run --with opencv-contrib-python python ../perception/live.py 0 1
uv run --with opencv-contrib-python python ../perception/live.py 0 1 --cameras cams.json

# sur des images, sans fenetre (ici les images brutes laissees par la demo)
uv run --with opencv-contrib-python python ../perception/live.py \
    ../perception/out/cam_left_raw.png ../perception/out/cam_right_raw.png --once

# verifier la calibration des cameras sur une photo (outil du simulateur)
uv run climbing-sim-cameras

# regenerer les images de ce document, puis le compiler
uv run --with opencv-contrib-python python ../perception/doc/make_figures.py
cd ../perception/doc && latexmk -xelatex perception_aruco.tex
```

B Référence des fonctions

Fonction	Rôle
`perception.py`	
`detect(image)`	marqueurs d'une image RGB : `{ID: 4 coins en pixels}`
`wall_marker_corners(id)`	les 4 coins du code d'un marqueur du mur, en 3D (repère mur)
`camera_pose(markers, K, dist)`	pose d'une caméra à partir des marqueurs 1–4 : `(rvec, tvec, rms)`
`marker_pose(views, size)`	pose d'un marqueur dans le repère du mur (triangulation, ou PnP avec une caméra)
`robot_pose(views, size, R, p)`	pose du tronc dans le repère du mur
`level_with_gravity(pose, g, p)`	la même, inclinaison corrigée par la gravité de l'IMU
`to_world(R, p)`	repère du mur → repère du simulateur
`pixel_classes(image)`	classe de chaque pixel : prise, mur ou inconnu
`hold_candidates()`, `hold_points()`	les places possibles des prises ; 200 points de la prise
`hold_pixels()`, `hold_votes()`	projection des places dans une caméra ; votes d'une image
`select_holds(hit, known, seen)`	les prises, d'après les votes (seuils, chevauchements, paires)
`FoundHold`	une prise trouvée : `hold` (`col`, `row`, `socket`), `socket_wall`, `color`, `score`, `seen`
`annotate(image, tracker, nom, pose, holds)`	image annotée (marqueurs, prises, repère du tronc)
`Tracker`	le tout : `calibrate()`, `find_holds()`, `reset_holds()`, `update(images, gravity)` ; `min_views` (2), `seen_by`
`robot_marker.py`	
`CODE`, `R_TRUNK_MARKER`, `P_TRUNK_MARKER`	le marqueur ID 5 de la fiche : 42 mm, axes, centre $(-33; 0; 54{,}75)$ mm
`MarkedSim(...)`	`ClimbingSim` avec l'autocollant sur le dos
`add_robot_marker(spec)`, `sticker_image()`	ajoute l'autocollant à une scène ; son image

C Petit glossaire

Pose
position et orientation d'un objet (6 nombres : 3 pour la position, 3 pour la rotation).

Intrinsèques
paramètres propres à une caméra (focale, centre de l'image, distorsion) ; ne changent pas quand on la déplace.

Extrinsèques
pose de la caméra par rapport au monde (ici, au mur) ; changent dès qu'on la bouge.

PnP
(*Perspective-n-Point*) retrouver la pose d'une caméra à partir de points 3D connus et de leurs pixels.

Erreur de reprojection
écart, en pixels, entre les points détectés et ceux recalculés avec la pose trouvée ; mesure la qualité d'une calibration.

Triangulation
retrouver un point 3D comme l'intersection des rayons de deux caméras.

Kabsch
méthode qui trouve la rotation et la translation superposant au mieux deux nuages de points.

IPPE
méthode de PnP exacte pour des points dans un même plan (`SOLVEPNP_IPPE`), et sa variante pour un marqueur carré seul (`SOLVEPNP_IPPE_SQUARE`).

IMU
centrale inertielle du robot : gyroscope et accéléromètre ; donne la direction de la gravité, donc son inclinaison, mais pas son cap.

Saturation
vivacité d'une couleur (0 : gris, 255 : couleur pure) ; elle sépare les prises du bois pâle.

Coupelle (socket)
le creux de la prise où se pose la boule au bout de chaque membre du robot.